

TP N°9 - MongoDB Parte I

Instalación del motor

Antes de comenzar si desean usar **dockers/contenedores**, sigan los siguientes pasos:

- Instalar Docker Desktop for Windows o for Mac o Docker Engine en Linux
- Para descargar la versión oficial de MongoDB: `docker pull mongo`
- Levantar el contenedor: `docker run --name Mymongo -p 27017:27017 -d mongo`
- Para levantar un Shell bash dentro del contenedor: `docker exec -it Mymongo bash`
- Para apagar el contenedor con: `docker stop Mymongo`
- Para iniciarla nuevamente: `docker start Mymongo`
- Para listar todos los containers existentes: `docker ps -a`
- Para copiar archivos del host al contenedor: `docker cp <filename> Mymongo:/<filename>`

Otra opción es instalar mongoDB, en su versión community, en la PC que vayan a utilizar para este práctico:

[Instalar MongoDB Community Edition - Manual de base de datos - MongoDB Docs](#)

Instalación del cliente

Una opción es utilizar el cliente de línea de comandos, para ello dentro de la terminal, ejecuten el comando `mongosh` para acceder a la Shell de MongoDB.

Otra opción es utilizar DataGrip, configurando el data source correspondiente. Si quieren utilizar una GUI diferente a DataGrip, pueden optar por alguna de estas:

- MongoDB Compass (oficial): <https://www.mongodb.com/products/tools/compass>
- Robo 3T is now Studio 3T Free (open-source) : <https://robomongo.org/download>

Introducción

MongoDB es una base de datos de documentos, de código abierto que ofrece alto rendimiento, alta disponibilidad y escalamiento automático.

Un registro en MongoDB es un **documento**, representado por una estructura de datos compuesta por pares de campo y valor. Los documentos en MongoDB poseen formato **JSON**. Los valores de los campos pueden incluir otros documentos, arreglos y arreglos de documentos.

```
{
  nombre: "Luis",
  edad: 34,
  estatus: "A",
  grupos: ["noticias", "deportes"]
}
```

← Campo: valor
← Campo: valor
← Campo: valor
← Campo: valor

COMANDOS BÁSICOS

En el cliente **mongosh** (cliente Shell), los comandos son ejecutados a través del objeto db. Por ejemplo:

- `db.help()`: Proporciona una lista de los comandos que se pueden ejecutar
- `db.stats()`: Proporciona estadísticas sobre la base de datos

NOTA: Al tratarse de una consola basada en JavaScript, si ejecuta un comando y omite los paréntesis (), se mostrará el cuerpo del método en lugar de la ejecución del mismo.

1. Probemos el comando `db.stats()` → que devuelve información estadística sobre la base a la cual están conectados

El método global **use** permite seleccionar la base de datos (No importa si la base de datos no existe aún, de hecho la primera colección que crearemos generará efectivamente la base de datos, en este caso *lab*):

2. Ejecutar `use lab`
3. Una vez seleccionada la base de datos, es posible consultar todas las colecciones que posee con `db.getCollectionNames()`

Al ejecutar el comando debe obtener un arreglo vacío []. No es necesario crear un esquema para la colección, puede simplemente insertar un nuevo documento en la nueva colección.

4. Use el comando **insertOne** para insertar un nuevo documento, suministrando el documento a insertar: `db.players.insertOne({name:'Aaron Appindangoye', height: 182, weight:187})`
5. Ejecutar nuevamente el comando `db.getCollectionNames()` → ahora debe aparecer la colección `players`
6. Usa el comando **find** contra `players` para retornar los documentos que posee la colección: `db.players.find()`

Adicional a los datos que se especificaron al insertar el documento, se observa un campo `_id`. Todo documento debe tener un campo `_id` único. Se puede asignar uno, o dejar que MongoDB genere automáticamente el ObjectId correspondiente.

7. Para agregar un nuevo documento a la colección ejecutar `db.players.insertOne({name:'Edinson Cavani', height: 182, dob: new Date(1987,2,14,0,0)})`
8. Ejecutar nuevamente `db.players.find()`
9. Eliminar los documentos de la colección `players` con el comando **deleteMany**. Si no indicas ningún selector, borrará todos los documentos: `db.players.deleteMany({})`
10. Ejecutar los siguientes inserts para tener más datos en la colección `players`:
`db.players.insertOne({name:'Luka Modric', height: 180, weight:143, dob: new Date (1985,9,9,0,0), preferred_foot: 'right', hobbies:['Playing football','Watching TV series','Swimming']});`
`db.players.insertOne({name:'Harry Kane', weight:143, dob: new Date (1993,7,28,0,0),preferred_foot: 'right', hobbies:['Watching Movies','Swimming']});`

```

db.players.insertOne({name:'Neymar', height: 175, weight:150, dob: new Date(1992,2,5,0,0),
preferred_foot: 'right', hobbies:['Video games','Watching TVseries','Swimming']});
db.players.insertOne({name:'David Silva', height: 170, weight:148, dob: new Date(1986,8,1,0,0),
preferred_foot: 'left', hobbies:['Playing football', 'Watching Movies']});
db.players.insertOne({name:'Eden Hazard', height: 172, weight:163, dob: new Date(1991,1,7,0,0),
preferred_foot: 'right', hobbies:['Watching Movies','Video games','Watching TV series']});
db.players.insertOne({name:'Antoine Griezmann', height: 175, weight:148, dob: new Date
(1991,3,21,0,0), preferred_foot: 'left', hobbies:['Video games','Swimming']});
db.players.insertOne({name:'Lionel Messi', height: 170, weight:159, dob: new Date(1987,6,24,0,0),
preferred_foot: 'left', hobbies:['Watching Movies','Video games','Watching TV series','Swimming']});
db.players.insertOne({name:'Cristiano Ronaldo', height: 185, weight:176, dob: new Date
(1985,2,5,0,0), preferred_foot: 'left', hobbies:['Video games','Swimming']});
db.players.insertOne({name:'Toni Kroos', height: 182, weight:172, dob: new Date(1990,1,4,0,0),
preferred_foot: 'right', hobbies:['Watching Movies','Video games','Watching TV series']});
db.players.insertOne({name:'Sergio Ramos', height: 182, weight:165, dob: new Date(1986,3,30,0,0),
preferred_foot: 'right', hobbies:['Video games','Watching TV series']});
db.players.insertOne({name:'Samuel Umtiti', height: 180, weight:165, dob: new
Date(1993,11,14,0,0), preferred_foot: 'left', hobbies:['Playing football','Swimming']});
db.players.insertOne({name:'Paulo Dybala', height: 175, weight:161, dob: new
Date(1993,11,15,0,0), preferred_foot: 'left', hobbies:['Playing football','Watching TV
series','Swimming']});

```

SELECTORES

Los selectores permiten consultar documentos particulares dentro de la colección:

- **{field: value}** es usado para encontrar cualquier documento donde field sea igual a value
- **{field1: value1, field2: value2}** permite realizar una declaración **and**
- Los selectores especiales **\$lt**, **\$lte**, **\$gt**, **\$gte** y **\$ne** son usados por las operaciones menor que, menor o igual que, mayor que, mayor o igual que y no igual respectivamente

11. Por ejemplo, para obtener todos los jugadores que tengan *preferred_foot* left y pesen más de 170 libras (*weight* > 170), se puede utilizar el siguiente comando:
db.players.find({preferred_foot: 'left', weight: {\$gt: 170}})

12. El operador **\$exists** es usado para determinar la presencia o ausencia de un campo, por ejemplo para ver que documentos de la colección *players* no posee el campo *height*, ejecutar: *db.players.find({height: {\$exists: false}})*

13. En caso de que se desee agregar una declaración con el operador OR, se utiliza **\$or** seguido de un arreglo con los valores que se desean evaluar:

```

db.players.find({preferred_foot: 'right',
                $or: [{hobbies: 'Playing football'},
                     {hobbies: 'Video games'},
                     {weight: {$lt: 150}}]}
})

```

EXPRESIONES REGULARES

MongoDB soporta expresiones regulares compatibles con Perl, para consultas basadas en patrones de coincidencia identificados en las cadenas de caracteres.

14. Consultar todos los jugadores cuyo nombre comience por la letra S: *db.players.find({name: { \$regex: "^S"}})*
15. Consultar todos los jugadores cuyo nombre termine con la letra o: *db.players.find({name: { \$regex: "o\$"}})*

COMANDOS DE ACTUALIZACIÓN

En su forma más simple, la operación update tiene dos argumentos: el selector (where) a usar y el campo del documento a actualizar (**\$set**).

16. Por ejemplo, el jugador que lleva por nombre Lionel Messi, ha subido de peso, por ello debemos actualizar el campo weight: *db.players.updateOne({name: 'Lionel Messi'}, {\$set: {weight: 180}})*
17. Adicionalmente a \$set, podemos aprovechar otros modificadores para hacer actualizaciones. Todos estos modificadores adicionales de update operan directamente sobre los campos del documento. Por lo tanto el documento completo no será reemplazado o borrado. Por ejemplo, el operador **\$inc** es usado para incrementar o decrementar de acuerdo al parámetro que se utilice (positivo o negativo, respectivamente).

Si a Luka Modric se le adjudicó incorrectamente un valor de 180 para el campo height, cuando debería ser 175, entonces ejecutar lo siguiente para corregirlo: *db.players.updateOne({name: 'Luka Modric'}, {\$inc: {height: -5}})*

18. Si David Silva de pronto desarrolla un gusto adicional por la natación, podemos agregarlo a su campo hobbies con el modificador **\$push** (por tratarse de un arreglo).

Primero ver los hobbies que tiene actualmente: *db.players.find({name: 'David Silva'})*
Luego realizar el push: *db.players.updateOne({name: 'David Silva'}, {\$push: {hobbies: 'Swimming'}})*

19. Los updates son compatibles con upserts. Un upsert actualiza el documento si lo encuentra o lo inserta si no lo encuentra. Los upserts son útiles de emplear en determinadas situaciones. Para habilitar upserting tenemos que especificar un tercer parámetro en true en el comando que utilicemos para el update.

Un ejemplo aplicable de este caso es un contador de visitas a un sitio web. Si se desea mantener un recuento total, se tendría que validar si el registro ya existe para la página, y basado en eso decidir si ejecutar un update o un insert.

Con el tercer parámetro omitido (o establecido en false), la ejecución de la siguiente instrucción no tendrá ningún resultado: *db.hits.updateOne({page: 'players'}, {\$inc: {hits: 1}});*

20. Sin embargo, para permitir upserts (tercer parámetro true) y lograr un resultado diferente:
db.hits.updateOne({page: 'players'}, {\$inc:{hits:1}}, {upsert:true});
21. Dado que no existen documentos con un campo *page* igual a *players*, se inserta un nuevo documento. Si se ejecuta el comando de nuevo, el documento existente se actualizará y *hits* se incrementará a 2. Ejecutar el comando anterior cuantas veces quieran y verificar la cantidad de *hits*.
22. Hasta el momento, en los ejemplos anteriores se ha actualizado un único documento. Sin embargo, si queremos actualizar varios documentos a la vez, ejecutar el siguiente comando:
db.players.updateMany({}, {\$set:{active:true}});

SELECCIÓN DE CAMPOS

El comando *find* posee un segundo parámetro opcional. Este parámetro es la lista de los campos que se desean obtener a la salida.

23. Si queremos obtener solo todos los nombres de *players* ejecutando:
db.players.find({}, {name:1});
24. Por defecto, el campo *_id* siempre es retornado. Podemos excluirlo explícitamente especificando: *db.players.find({}, {name:1, _id:0});*
25. El método **sort** permite ordenar los resultados por los campos que se indiquen. Se utiliza 1 para orden ascendente y -1 para descendente. Por ejemplo, podemos consultar los jugadores más altos primero: *db.players.find({}, {name:1, height:1, _id:0}).sort({height:-1})*
26. También es posible paginar los resultados, a través de los métodos **limit** y **skip**. Para obtener solamente el segundo y tercer jugador más pesado, ejecuta el siguiente comando:
db.players.find().sort({weight:-1}).limit(2).skip(1)

Usando *limit* en conjunto con *sort*, es una buena forma de evitar problemas de performance cuando hacemos ordenamiento en campo sin índices.

27. Se puede ejecutar **count** directamente en una colección, para conocer el número de documentos que cumplen con determinada condición:
db.players.countDocuments({hobbies:'Swimming'})
db.players.find({hobbies:'Swimming'}).count()

DOCUMENTOS EMBEBIDOS

En MongoDB el valor de un campo puede ser otro documento JSON, esto es útil por ejemplo para simular JOINS.

28. Incorporémosle al jugador Cristiano Ronaldo, un documento con información correspondiente a su equipo: `db.players.updateOne({name: 'Cristiano Ronaldo'},{$set:{team:{team_long_name: 'Juventus', team_short_name: 'JUV'}}})`
29. Los documentos embebidos también pueden ser accedidos usando la notación de punto: `db.players.find({'team.team_short_name': 'JUV'})`

ÍNDICES Y ADMINISTRACIÓN

Los índices en MongoDB funcionan muy parecido a los índices en una base de datos relacional, ayudan a mejorar el rendimiento de los queries y ordenados.

30. Los índices son creados con el comando **ensureIndex**: `db.players.ensureIndex({name:1})`
31. Los índices pueden ser borrados usando **dropIndex**: `db.players.dropIndex({name:1})`
32. Un índice único se crea utilizando el parámetro **unique**: `db.players.ensureIndex({name:1},{unique:true})`
33. Los índices pueden ser creados en campos embebidos (usando la notación de punto) y en campos de arreglos. También, es posible crear índices compuestos: `db.players.ensureIndex({name:1, weight:1})`
34. Para validar si los queries están usando índices o no, se puede utilizar el método `explain`, al final de la consulta.

Si se mantiene el índice del punto 32, veremos con la siguiente query que figura el nombre del índice que es usado: `db.players.find({name: 'Lionel Messi'}).explain()`

Ejercicios:

- 1) Construya en MongoDB la colección o las colecciones necesarias, para representar los siguientes datos, correspondientes a las Bandas de músicos jóvenes de la Ciudad de Buenos Aires.

Hint: antes de definir el modelo revise las consultas que se piden resolver más abajo.

NOMBRE DEL SOLISTA	GENERO	ESTILO	FECHA_INSCRIPCION	DISCOS	BARRIO	INTEGRANTES
VER K BITCH	INSTRUMENTAL		20/01/2018	DIOS AGUJERO NEGRO (2008); APICHONADOS (2007); MERIDIANO (2006); CAUSALIDADES (2005); PLANETA ESMERALDA (2001)	VERSALLES	1
TROTAMUNDOS	INDIE		30/11/2017	HECHO BOLITA (2014)	VILLA LURO	4
EFFECTO ALFONS	ROCK	POWER TRIO	27/11/2017	EFFECTO ALFONS (2000)	BARRACAS	3
MARCELO GIULLITTI	SOLISTA		26/11/2017		AGRONOMIA	1
AFTERLIFE	ROCK	ROCK ALTERNATIVO	14/11/2017		BALVANERA	5
VIRGINIA FERREYRA	ROCK	ROCK POP	14/11/2017		VILLA DEL PARQUE	1
LMV	POP		10/11/2017		LA LUCILA	6
EFFECTO ALFONS	ROCK	POWER TRIO	06/11/2017	EFFECTO ALFONS (1995)	BARRACAS	3
TANTAS PREGUNTAS	PUNK	PUNK ROCK	27/10/2017	DESPUES DE TODO (2006); LIBRE ALBEDRIO (2006)	MORENO	3
TAL VEZ DE PASO	POP	POP ROCK	26/10/2017		PUERTO MADERO	4
LA SURTIDA FOLCK	FOLKLORE		25/10/2017		BERAZATEGUI	7
JAYDEE M	HIP HOP / RAP		24/10/2017		BARRACAS	1

Luego, ejecute los comandos que permitan obtener los siguientes resultados:

- 2) Todas las bandas en orden de menor a mayor de acuerdo al número de integrantes
- 3) Las dos bandas con mayor número de integrantes
- 4) Sumar un integrante más a todas las bandas
- 5) Bandas cuyo género es Rock
- 6) Bandas cuyo género es Rock o su estilo tiene algo de Rock
- 7) Bandas que han lanzado disco en el año 2006
- 8) Bandas que han lanzado disco después del 2010
- 9) Cantidad de Bandas en el barrio Barracas
- 10) Promedio de integrantes por género en orden alfabético, usando una vista llamada *bandas_resumen* (que deberán crear) que tenga solo los datos del solista, el género de la banda, el barrio y los integrantes
- 11) Cantidad de bandas por barrio, usando la vista creada en el punto anterior, en orden de barrios más musicales a menos